



《AI大模型Ragent项目》——知识库答不了的问题，交给MCP工具去查

来自： 拿个offer-开源&项目实战



马丁

2026年05月14日 21:56

开篇引言

上一篇拆解了后处理流水线——三个通道返回的约 30 条原始 Chunk，经过去重、Cross-Encoder 精排、topK 截断，最终只留 5 条高质量上下文喂给大模型。到那里为止，KB 检索链路就完整收尾了：意图识别告诉系统该查哪个知识库，多通道检索把文档捞出来，后处理流水线把结果精炼到 5 条。

但 KB 检索不是万能的。

还是电商客服的场景。用户问“AirPods Pro 怎么退货”——知识库里有退货政策文档，KB 检索能搞定。但如果用户问的是“我上周提的退货申请现在到哪一步了”呢？这条退货申请的实时状态不会写在任何知识库文档里，它在订单系统的数据库里，每小时都可能变。类似的场景还有很多：查物流轨迹、查优惠券余额、查维修工单进度……这些数据不在知识库里，而是在企业的业务系统里。

要回答这类问题，得让系统实时去业务系统查。这就是 MCP（Model Context Protocol，模型上下文协议）工具调用要解决的事。

本篇不讲 MCP 协议本身是什么——基础系列已经讲过 Function Call 和 MCP 的概念，MCP 系列也有专门的协议规范和 SDK 架构拆解。本篇只聚焦一件事：在 Ragent 这个系统里，一次 MCP 工具调用从被识别到被执行再到结果回流，完整链路是怎么跑的。

MCP 在八阶段里的位置

1. 回到全景地图

先回到第 1 篇的全景地图，定位一下 MCP 在整个流水线中的位置：

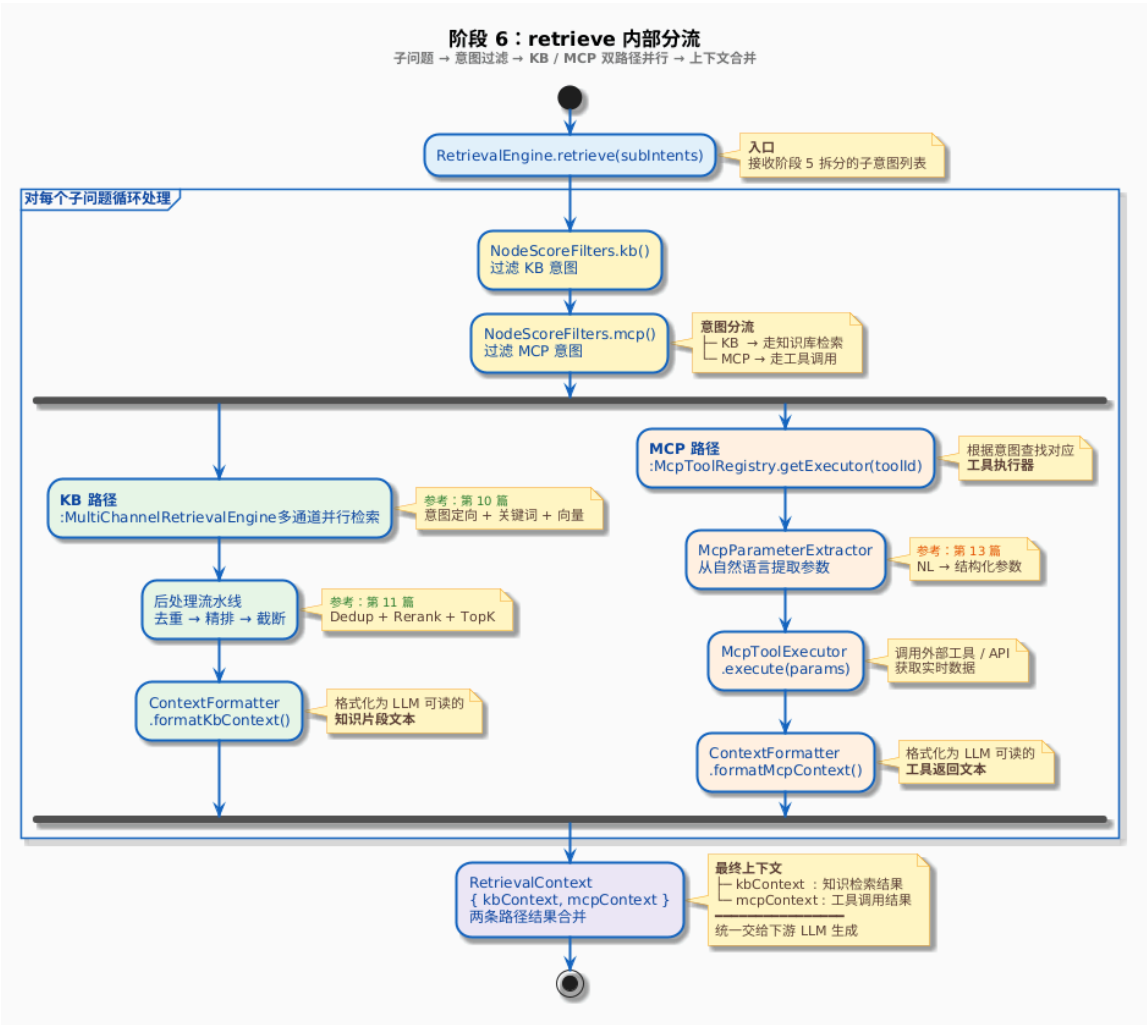
```
StreamChatPipeline.execute(ctx)
```

阶段 1: loadMemory	— 加载会话记忆
阶段 2: rewriteQuery	— 查询改写与子问题拆分
阶段 3: resolveIntents	— 意图识别
阶段 4: handleGuidance	— 歧义引导 [短路点 #1]
阶段 5: handleSystemOnly	— 系统直答 [短路点 #2]
阶段 6: retrieve	— 多通道检索 (KB + MCP) ← MCP 在这里
阶段 7: handleEmptyRetrieval	— 空结果兜底 [短路点 #3]
阶段 8: streamRagResponse	— Prompt 组装 + 流式生成

MCP 不是一条独立的流水线。它嵌在阶段 6 retrieve 内部，和 KB 检索并行执行。意图识别阶段（阶段 3）已经判断出哪些子问题需要查知识库、哪些需要调工具，到了阶段 6 就各走各的路——KB 意图走多通道检索 + 后处理流水线（第 10~11 篇的内容），MCP 意图走工具注册表查找 + 参数提取 + 远程执行。两条路的结果最终汇合到同一个 RetrievalContext 里，一起传给阶段 8 做 Prompt 组装。

2. 阶段 6 内部的分流

下面这张图把 RetrievalEngine.retrieve() 内部的 KB/MCP 并行分流画出来：



KB 路径在第 10~11 篇已经完整拆过了。接下来按 MCP 路径的顺序，一步步拆解。

意图分流：KB 和 MCP 怎么分开的

分流发生在 RetrievalEngine.buildSubQuestionContext() 方法的前两行——第 9 篇讲意图到检索映射时已经见过这段代码，这里只做简要回顾：

```
private SubQuestionContext buildSubQuestionContext(SubQuestionIntent intent, int topK) {
    // 按 IntentKind 把 KB 和 MCP 意图分开
    List<NodeScore> kbIntents = NodeScoreFilters.kb(intent.nodeScores());
    List<NodeScore> mcpIntents = NodeScoreFilters.mcp(intent.nodeScores());

    KbResult kbResult = retrieveAndRerank(intent, kbIntents, topK);

    String mcpContext = CollUtil.isEmpty(mcpIntents)
        ? executeMcpAndMerge(intent.subQuestion(), mcpIntents)
        : "";

    return new SubQuestionContext(
        intent.subQuestion(), kbResult.groupedContext(), mcpContext, kbResult.intentChunks()
    );
}
```

NodeScoreFilters.mcp() 的过滤条件很明确——节点非空、kind 等于 IntentKind.MCP、mcpToolId 非空：

```
public static List<NodeScore> mcp(List<NodeScore> scores) {
    return scores.stream()
        .filter(ns -> ns.getNode() != null && ns.getNode().isMCP())
        .filter(ns -> StrUtil.isNotBlank(ns.getNode().getMcpToolId()))
}
```

```
        .toList();
    }
}
```

分流的关键在于 IntentNode 上的几个字段。每个意图节点在配置时就声明了自己的类型：

```
public enum IntentKind {
    KB(0),      // 知识库类，走 RAG 检索
    SYSTEM(1),  // 系统交互类，如欢迎语
    MCP(2);     // MCP 工具调用，查实时数据
}
```

当一个意图节点的 kind 被配置为 MCP 时，它还会携带几个 MCP 专属字段：

字段	含义	示例
mcpToolId	工具 ID，关联注册表中的执行器	order_query
paramPromptTemplate	自定义参数提取提示词（可选）	针对特定工具的定制提取逻辑
promptSnippet	回答规则片段（可选）	回答时附上订单编号和物流公司信息

继续用电商客服的场景。假设意图树里有一个节点叫订单查询，配置长这样：

字段	值
id	biz-order-query
name	订单查询
kind	MCP
mcpToolId	order_query
description	查询用户的订单状态、物流轨迹、退货进度等实时信息

当用户问“我上周提的退货申请到哪一步了”，意图识别命中了这个节点，NodeScoreFilters.mcp() 就会把它过滤出来。后续流程知道：这不是去知识库搜文档，而是去调用 ID 为 order_query 的 MCP 工具。
那 order_query 这个字符串怎么对应到一个可以真正执行的工具？

工具注册表：mcpToolId 怎么对应到可执行的工具

1. McpToolRegistry 接口

McpToolRegistry（MCP 工具注册表）定义了工具注册和查找的核心能力：

```
public interface McpToolRegistry {

    void register(McpToolExecutor executor);           // 注册工具执行器
    void unregister(String toolId);                   // 注销工具
    Optional<McpToolExecutor> getExecutor(String toolId); // 按 ID 查找执行器
    List<Tool> listAllTools();                         // 列出所有工具定义
    boolean contains(String toolId);                   // 检查工具是否已注册
    int size();                                        // 已注册工具数量
}
```

接口很直接——核心就是 register() 和 getExecutor()。注册时传入一个执行器，查找时传入一个 toolId。

2. DefaultMcpToolRegistry 实现

默认实现 DefaultMcpToolRegistry 的内部结构就是一个 Map<String, McpToolExecutor>：

```
@Slf4j
@Component
```

```
@RequiredArgsConstructor
public class DefaultMcpToolRegistry implements McpToolRegistry {

    // 核心存储: toolId → 执行器
    private final Map<String, McpToolExecutor> executorMap = new HashMap<>();

    // Spring 容器中所有 McpToolExecutor Bean (自动注入)
    private final List<McpToolExecutor> autoDiscoveredExecutors;

    @PostConstruct
    public void init() {
        for (McpToolExecutor executor : autoDiscoveredExecutors) {
            register(executor);
        }
        log.info("MCP 工具自动注册完成, 共注册 {} 个工具", autoDiscoveredExecutors.size());
    }

    @Override
    public void register(McpToolExecutor executor) {
        String toolId = executor.getToolId();
        McpToolExecutor existing = executorMap.put(toolId, executor);
        if (existing != null) {
            log.warn("工具 {} 已存在, 已覆盖", toolId);
        } else {
            log.info("MCP 工具注册成功, toolId: {}", toolId);
        }
    }

    @Override
    public Optional<McpToolExecutor> getExecutor(String toolId) {
        return Optional.ofNullable(executorMap.get(toolId));
    }

    // ...
}
```

@PostConstruct 在应用启动时自动执行，把 Spring 容器中所有实现了 McpToolExecutor 接口的 Bean 注册进 Map。运行时通过 getExecutor(toolId) 查表就能找到对应的执行器。

但这些执行器从哪来？注册表自己只管存，不管造。执行器的创建靠另一个组件——McpClientAutoConfiguration。

3. 远程工具怎么注册进来的

在 Ragent 中，MCP 工具部署在独立的 MCP Server 上（比如订单服务跑在 localhost:8081，客服工单服务跑在 localhost:8082）。McpClientAutoConfiguration 负责在应用启动时连接这些 Server，发现它们暴露的工具，为每个工具创建执行器并注册。

先看配置。application.yaml 里声明了要连接的 MCP Server 列表：

```
rag:
  mcp:
    servers:
      - name: order-server
        url: http://localhost:8081/mcp
```

对应的配置类：

```
@Data
@ConfigurationProperties(prefix = "rag.mcp")
public class McpClientProperties {

    private List<ServerConfig> servers = new ArrayList<>();

    @Data
    public static class ServerConfig {
        private String name; // 服务名称
    }
}
```

```
        private String url;    // 服务地址
    }
}
```

再看启动时的自动注册逻辑：

```
@Slf4j
@Configuration
@RequiredArgsConstructor
@EnableConfigurationProperties(McpClientProperties.class)
public class McpClientAutoConfiguration {

    private final McpClientProperties properties;
    private final McpToolRegistry toolRegistry;
    private final List<McpSyncClient> clients = new ArrayList<>();

    @PostConstruct
    public void init() {
        List<McpClientProperties.ServerConfig> servers = properties.getServers();
        if (servers == null || servers.isEmpty()) {
            log.info("未配置 MCP Server, 跳过远程工具注册");
            return;
        }
        for (McpClientProperties.ServerConfig server : servers) {
            registerRemoteTools(server);
        }
    }

    private void registerRemoteTools(McpClientProperties.ServerConfig server) {
        try {
            // 1. 建立 HTTP 连接
            String mcpUrl = server.getUrl().endsWith("/mcp")
                ? server.getUrl() : server.getUrl() + "/mcp";
            HttpClientStreamableHttpTransport transport =
                HttpClientStreamableHttpTransport.builder(mcpUrl).build();

            // 2. 初始化 MCP 客户端, 完成协议握手
            McpSyncClient client = McpClient.sync(transport)
                .clientInfo(new Implementation("ragent-bootstrap", "1.0.0"))
                .build();
            client.initialize();
            clients.add(client);

            // 3. 发现远端暴露的所有工具
            ListToolsResult result = client.listTools();
            List<Tool> tools = result.tools();

            // 4. 为每个工具创建执行器, 注册进注册表
            for (Tool tool : tools) {
                McpClientToolExecutor executor = new McpClientToolExecutor(client, tool);
                toolRegistry.register(executor);
            }
            log.info("MCP Server [{}] 返回 {} 个工具", server.getName(), tools.size());
        } catch (Exception e) {
            log.error("连接 MCP Server [{}] 失败, 跳过工具注册, reason={}",
                server.getName(), e.getMessage());
        }
    }

    @PreDestroy
    public void destroy() {
        for (McpSyncClient client : clients) {
            try { client.close(); } catch (Exception e) { log.warn("关闭 MCP 客户端失败", e); }
        }
    }
}
```

```
        }
    }
}
```

整个过程在应用启动阶段完成，时序是这样的：

- 1. 读取 application.yaml 中配置的 MCP Server 列表
- 2. 对每个 Server，通过 Streamable HTTP 建立连接，调用 client.initialize() 完成 MCP 协议握手
- 3. 调用 client.listTools() 获取该 Server 暴露的所有工具定义——Tool 对象包含工具名称、描述、参数 Schema
- 4. 为每个 Tool 创建一个 McpClientToolExecutor（持有 McpSyncClient 引用和 Tool 定义），注册进 McpToolRegistry

启动完毕后，注册表里就有了所有可用工具的执行器。比如 order-server 暴露了一个 order_query 工具，注册表里就多了一条 "order_query" → McpClientToolExecutor 的映射。后续运行时通过 toolId 一查就能找到。

注意 catch 块的设计：如果某个 MCP Server 连接失败（比如还没启动），McpClientAutoConfiguration 会打日志并跳过，不影响其他 Server 的注册，也不影响应用启动。MCP 工具是增强能力，不能因为一个工具服务挂了就让整个系统起不来。

@PreDestroy 则负责应用关闭时释放所有客户端连接，避免资源泄漏。

工具执行器：execute 方法做了什么

1. McpToolExecutor 接口

注册表查到了执行器，接下来就是调用。先看执行器的接口定义：

```
public interface McpToolExecutor {

    // 工具元信息 (MCP 官方 SDK 的 Tool 类型)
    Tool getToolDefinition();

    // 执行工具调用
    CallToolResult execute(Map<String, Object> parameters);

    // 工具 ID (默认取 Tool.name())
    default String getToolId() {
        return getToolDefinition().name();
    }
}
```

接口很干净。execute() 传入参数 Map，返回 CallToolResult。参数怎么来的先不管（下一篇讲），这里关注执行器怎么把参数发出去、把结果拿回来。

2. McpClientToolExecutor：远程调用的实现

McpClientToolExecutor 是最核心的执行器实现，负责通过 MCP 协议调用远端 Server：

```
@Slf4j
@RequiredArgsConstructor
public class McpClientToolExecutor implements McpToolExecutor {

    private final McpSyncClient mcpClient;
    private final Tool toolDefinition;

    @Override
    public CallToolResult execute(Map<String, Object> parameters) {
        long startMs = System.currentTimeMillis();
        try {
            Map<String, Object> args = parameters != null ? parameters : Map.of();
            // 通过 MCP SDK 发起远程调用
            CallToolResult result = mcpClient.callTool(
                new CallToolRequest(toolDefinition.name(), args)
            );
        } catch (Exception e) {
            log.error("MCP tool call failed: {}", e.getMessage());
            return CallToolResult.failure(e.getMessage());
        }
        return result;
    }
}
```

```
    );
    log.info("MCP 远程工具调用完成, toolId={}, params={}, contentSize={}, elapsed={}ms",
            toolDefinition.name(), args,
            result.content() != null ? result.content().size() : 0,
            System.currentTimeMillis() - startMs);

    return result;
} catch (Exception e) {
    String reason = e.getMessage() != null ? e.getMessage() : e.getClass().getSimpleName();
    log.warn("MCP 远程工具调用异常, toolId={}, elapsed={}ms, reason={}",
            toolDefinition.name(), System.currentTimeMillis() - startMs, reason);
    // 异常不上抛，统一包成错误结果
    return CallToolResult.builder()
        .content(List.of(new TextContent("远程调用失败: " + reason)))
        .isError(true)
        .build();
}
}
```

几个设计要点值得注意。
用的是官方 SDK 的 CallToolResult，不自定义包装类。CallToolResult 来自 MCP Java SDK (io.modelcontextprotocol.spec.McpSchema)，它的核心就两个字段：

字段	类型	含义
content	List<Content>	返回内容列表，通常是 TextContent（文本内容）
isError	Boolean	是否为错误结果

异常不上抛，统一包成错误结果。远程调用可能因为网络超时、Server 内部异常等各种原因失败。catch 块里把异常转成一个 isError=true 的 CallToolResult，附带错误原因文本。这样上层代码不需要处理异常分支，所有结果都是 CallToolResult，只需检查 isError 标志。

每次调用都记录耗时。elapsed={}ms 打在日志里，生产环境排查性能问题靠这些指标。

3. 两层异常兜底

工具执行的异常处理设计了两层保护。第一层刚看过了——McpClientToolExecutor.execute() 内部的 catch 把异常转成错误结果。第二层在 RetrievalEngine.executeMcpTools() 方法的外层：

```
List<CompletableFuture<ToolOutput>> futures = mcpIntentScores.stream()
    .map(ns -> CompletableFuture.supplyAsync(
        () -> {
            String toolId = ns.getNode().getMcpToolId();
            try {
                CallToolResult result = executeSingleMcpTool(question, ns.getNode());
                return result == null ? null : new ToolOutput(toolId, result);
            } catch (Exception e) {
                // 即使执行器内部的 catch 没兜住，这里还有一层
                log.error("MCP 工具调用异常, toolId: {}", toolId, e);
                return new ToolOutput(toolId, CallToolResult.builder()
                    .content(List.of(new TextContent("工具调用异常: " + e.getMessage())))
                    .isError(true)
                    .build());
            }
        },
        mcpBatchExecutor
    ))
    .toList();
```

两层保护的目的一样：一个工具调用失败，不能拖垮整个问答流程。这个设计思路和第 10 篇多通道检索的容错策略思路一致——单点故障降级，不让整个链路崩溃。

完整执行链路：从意图命中到结果返回

1. executeSingleMcpTool：单个工具的调用入口

把各个环节串起来。executeSingleMcpTool() 是单个工具调用的入口：

```
private CallToolResult executeSingleMcpTool(String question, IntentNode intentNode) {
    // 第一步：从意图节点取出 toolId
    String toolId = intentNode.getMcpToolId();

    // 第二步：去注册表查执行器
    Optional<McpToolExecutor> executorOpt = mcpToolRegistry.getExecutor(toolId);
    if (executorOpt.isEmpty()) {
        log.warn("MCP 工具不存在: {}", toolId);
        return null;
    }

    McpToolExecutor executor = executorOpt.get();
    Tool tool = executor.getToolDefinition();

    // 第三步：从自然语言中提取工具参数（下一篇详细讲）
    String customParamPrompt = intentNode.getParamPromptTemplate();
    Map<String, Object> params = mcpParameterExtractor.extractParameters(
        question, tool, customParamPrompt
    );

    // 第四步：执行工具调用
    return executor.execute(params != null ? params : new HashMap<>());
}
```

四步走完：intentNode.getMcpToolId() → mcpToolRegistry.getExecutor(toolId) → mcpParameterExtractor.extractParameters() → executor.execute(params)。

第三步的参数提取本篇先当黑盒——传入用户的自然语言和工具的参数 Schema，输出一个参数 Map。用户说“我上周提的退货申请到哪一步了”，参数提取器会从中抽出订单号或退货申请的关键信息，组装成工具需要的参数 Map。具体怎么做的，下一篇拆解。

这里提一个设计细节：意图节点上有一个 paramPromptTemplate 字段，允许为特定工具定制参数提取的提示词。比如订单查询工具和物流查询工具的参数提取，可能需要不同的 LLM 提示策略。这个定制化能力是通过意图节点配置实现的，不需要改代码。

2. executeMcpTools：多工具并行调用

如果一个子问题命中了多个 MCP 意图（比如用户问“帮我查一下订单 2024112801 的物流和退货进度”，同时命中了 logistics_query 和 return_query 两个工具），executeMcpTools() 会并行调用所有工具：

```
private Map<String, List<CallToolResult>> executeMcpTools(String question,
                                                            List<NodeScore> mcpIntentScores) {

    // 为每个 MCP 意图创建异步任务
    List<CompletableFuture<ToolOutput>> futures = mcpIntentScores.stream()
        .map(ns -> CompletableFuture.supplyAsync(
            () -> {
                String toolId = ns.getNode().getMcpToolId();
                try {
                    CallToolResult result = executeSingleMcpTool(question, ns.getNode());
                    return result == null ? null : new ToolOutput(toolId, result);
                } catch (Exception e) {
                    log.error("MCP 工具调用异常, toolId: {}", toolId, e);
                    return new ToolOutput(toolId, CallToolResult.builder()
                        .content(List.of(new TextContent("工具调用异常: " + e.getMessage()
                            .isError(true)
                            .build())))
                        .build());
                }
            }
        ))
        .toList();

    // 等待所有任务完成并聚合结果
    Map<String, List<CallToolResult>> results = new HashMap<>();
    for (ToolOutput toolOutput : futures.stream().map(CompletableFuture::join).toList()) {
        results.put(toolOutput.getId(), new ArrayList<>());
    }
    for (ToolOutput toolOutput : futures.stream().map(CompletableFuture::join).toList()) {
        results.get(toolOutput.getId()).add(toolOutput.getResult());
    }

    return results;
}
```



```

        },
        mcpBatchExecutor // 使用专用线程池
    ))
    .toList();

// 等待所有工具执行完毕，按 toolId 分组
return futures.stream()
    .map(CompletableFuture::join)
    .filter(Objects::nonNull)
    .collect(Collectors.groupingBy(
        ToolOutput::toolId,
        Collectors.mapping(ToolOutput::result, Collectors.toList())
    ));
}

private record ToolOutput(String toolId, CallToolResult result) {}

```

这段代码的结构和第 10 篇多通道检索的并行调度很像——`CompletableFuture.supplyAsync()` 提交到线程池，`join()` 等待所有结果。区别在于线程池用的是 `mcpBatchExecutor` (MCP 专用)，不和 KB 检索的 `ragRetrievalExecutor` 争资源。

结果按 `toolId` 分组存入 `Map<String, List<CallToolResult>>`。分组是为了后续格式化时能把每个工具的结果关联到对应的意图节点。

3. 结果格式化：formatMcpContext

工具执行完毕，`CallToolResult` 里的内容需要格式化成可以嵌入 Prompt 的文本。这个工作由 `ContextFormatter.formatMcpContext()` 完成：

```

@Override
public String formatMcpContext(Map<String, List<CallToolResult>> toolResults,
                               List<NodeScore> mcpIntents) {
    if (CollUtil.isEmpty(toolResults)) return "";

    // 建立 toolId → 意图节点的映射
    Map<String, IntentNode> toolToIntent = new LinkedHashMap<>();
    for (NodeScore ns : mcpIntents) {
        IntentNode node = ns.getNode();
        if (node != null && StrUtil.isNotBlank(node.getMcpToolId())) {
            toolToIntent.putIfAbsent(node.getMcpToolId(), node);
        }
    }

    // 对每个工具的结果做格式化
    return toolToIntent.entrySet().stream()
        .map(entry -> {
            List<CallToolResult> results = toolResults.get(entry.getKey());
            if (CollUtil.isEmpty(results)) return "";

            IntentNode node = entry.getValue();
            String snippet = StrUtil.emptyIfNull(node.getPromptSnippet()).trim();
            String body = mergeResultsToText(results);
            if (StrUtil.isBlank(body)) return "";

            // 如果意图节点配置了回答规则，渲染 rules 区块
            String snippetSection = StrUtil.isNotBlank(snippet)
                ? templateLoader.renderSection(CONTEXT_FORMAT_PATH,
                    "mcp-intent-rules", Map.of("rules", snippet))
                : "";
            // 渲染 mcp-section 模板
            return templateLoader.renderSection(CONTEXT_FORMAT_PATH, "mcp-section", Map.of(
                "snippet_section", snippetSection,
                "body", body
            ));
        })
    };
}

```

```
        .filter(StrUtil::isNotBlank)
        .collect(Collectors.joining("\n\n"));
    }
```

格式化用了 context-format.st 模板中的 mcp-section 区块：

```
--- section: mcp-section ---
{snippet_section}<data>
{body}
</data>
```

如果意图节点配置了 promptSnippet（回答规则片段），还会在前面加上 mcp-intent-rules 区块：

```
--- section: mcp-intent-rules ---
<rules>
{rules}
</rules>
```

用电商客服的订单查询举个例子，最终渲染出来的 MCP 上下文可能长这样：

```
<rules>
回答订单相关问题时，附上订单编号和当前状态，物流信息精确到最新一条。
</rules>
<data>
订单编号：2024112801
商品：AirPods Pro（第二代）
订单状态：退货处理中
退货进度：已收到退回商品，质检中，预计 1~2 个工作日完成退款
物流轨迹：2026-04-28 14:30 已签收（退货仓库）
</data>
```

3.1 成功和失败结果的区分处理

mergeResultsToText() 在合并多条 CallToolResult 时，会区分成功和失败的结果：

```
private String mergeResultsToText(List<CallToolResult> results) {
    List<String> successTexts = new ArrayList<>();
    List<String> errorTexts = new ArrayList<>();

    for (CallToolResult result : results) {
        boolean isError = result.isError() != null && result.isError();
        String text = extractTextContent(result);
        if (!isError && text != null) {
            successTexts.add(text);
        } else if (isError && text != null) {
            errorTexts.add("- 工具调用失败: " + text);
        }
    }

    StringBuilder sb = new StringBuilder();
    for (String text : successTexts) {
        sb.append(text).append("\n\n");
    }
    if (CollUtil.isNotEmpty(errorTexts)) {
        String errorList = String.join("\n", errorTexts);
        sb.append(templateLoader.renderSection(CONTEXT_FORMAT_PATH, "mcp-error",
            Map.of("error_list", errorList)));
    }
    return sb.toString().trim();
}
```

成功结果直接拼接，失败结果用 `<errors>` 标签包装。这样大模型在生成回答时能看到哪些工具调用成功了、哪些失败了，可以据此给出合理的回复——比如“物流信息查询成功，但退货系统暂时无法访问”。

4. 结果汇入主流水管线

MCP 上下文格式化完成后，和 KB 上下文一起装进 `RetrievalContext`：

```
@Data
@Builder
public class RetrievalContext {

    private String mcpContext;    // MCP 工具结果（格式化后的文本）
    private String kbContext;    // KB 检索结果（格式化后的文本）
    private Map<String, List<RetrievedChunk>> intentChunks; // 原始 Chunk（用于引用溯源）

    public boolean hasMcp() { return StrUtil.isNotBlank(mcpContext); }
    public boolean hasKb() { return StrUtil.isNotBlank(kbContext); }
    public boolean isEmpty() { return !hasMcp() && !hasKb(); }
}
```

`RetrievalContext` 是 KB 和 MCP 两条路径的汇合点。`mcpContext` 和 `kbContext` 是两个独立的 `String` 字段，各自携带格式化好的文本。到了阶段 8 `streamRagResponse`，`Pipeline` 在构建 LLM 请求时，会根据是否有 MCP 上下文调整模型参数：

```
ChatRequest chatRequest = ChatRequest.builder()
    .messages(messages)
    .temperature(ctx.hasMcp() ? 0.3D : 0D)
    .topP(ctx.hasMcp() ? 0.8D : 1D)
    .build();
```

MCP 场景下 `temperature` 设为 0.3 而不是 0。原因是工具返回的结构化数据（订单状态、物流轨迹等）需要模型做一定的整理和表述转换，完全确定性的生成反而可能太刻板。纯 KB 场景则保持 0，让模型严格忠实于检索到的文档内容。

另外，如果用户的问题被拆分成多个子问题，每个子问题的 MCP 结果会用 `sub-question-mcp-wrapper` 模板包装，带上编号和子问题文本：

```
<result index="1">
<question>订单 2024112801 的物流到哪了? </question>
<data>...</data>
</result>
```

单子问题时不包装，直接用原始上下文。

一个完整的例子：查询华东区销售数据

前面用电商客服的订单查询来讲概念，接下来用 Ragent 代码中真实存在的 sales_query 工具走一遍完整流程。这个工具由 mcp-server 模块的 SalesMcpExecutor 实现，支持按地区、时间、产品等维度查询销售数据——虽然不是电商场景，但 MCP 工具调用的链路完全一样，换成订单查询工具也是同一套代码在跑。

用户输入：“华东区本月的销售额是多少？”

阶段 1~3（记忆 → 改写 → 意图识别）：意图识别命中了意图树中的销售数据查询节点，kind=MCP，mcpToolId=sales_query，score=0.90。

阶段 6 进入 RetrievalEngine：

- NodeScoreFilters.mcp() 过滤出 MCP 意图，得到一个 NodeScore，节点的 mcpToolId 是 sales_query
- 进入 executeSingleMcpTool():

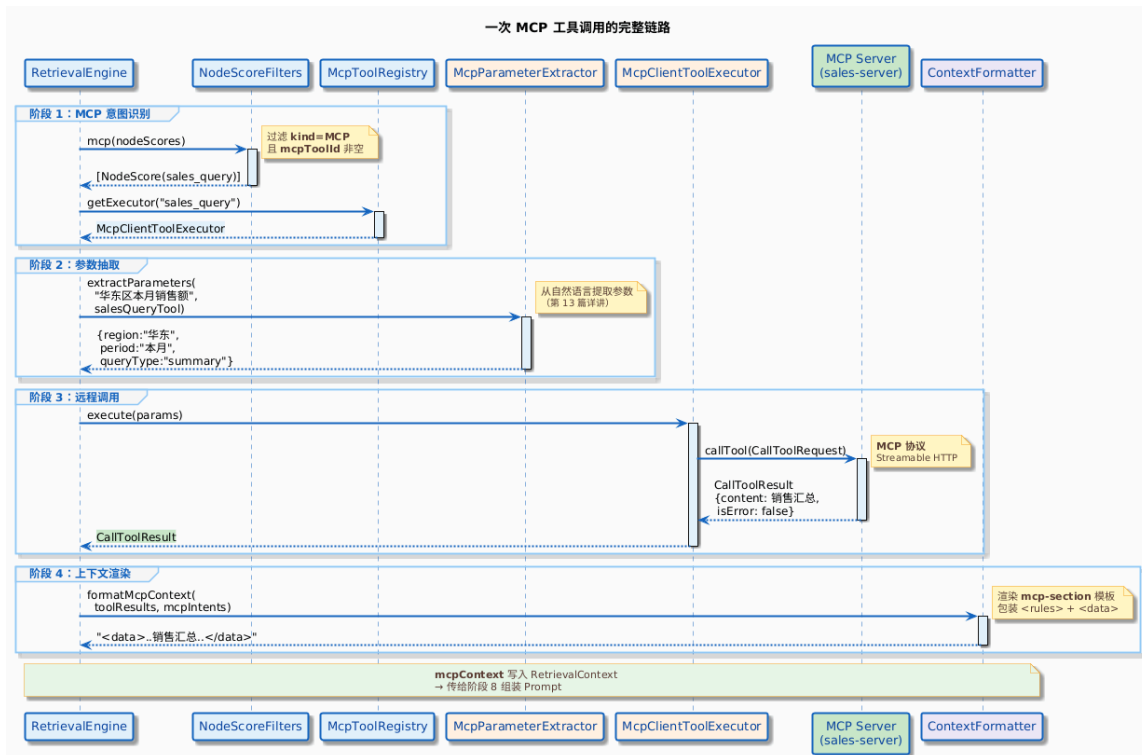
mcpToolRegistry.getExecutor("sales_query") 从注册表找到执行器——一个在启动时注册的 McpClientToolExecutor

mcpParameterExtractor.extractParameters() 从自然语言中提取出参数：{region: "华东", period: "本月", queryType: "summary"}

executor.execute(params) 通过 MCP SDK 调用远端的 sales-server
- Server 端处理：SalesMcpExecutor.handleCall() 收到请求后，解析参数，生成本月的销售数据，按华东地区筛选，按 summary 类型输出汇总报告，返回 CallToolResult (isError=false)
- ContextFormatter.formatMcpContext() 将返回的销售数据包装进 <data> 标签

阶段 8：RetrievalContext.hasMcp() 返回 true，Pipeline 选择 MCP 相关的 System Prompt 模板，temperature=0.3，大模型基于这份实时数据生成最终答案。

下面这张时序图把整个过程串起来：



Server 端的 SalesMcpExecutor 定义了工具的参数 Schema 和处理逻辑：

```
private Tool buildTool() {
    Map<String, Object> properties = new LinkedHashMap<>();
    properties.put("region", Map.of(
        "type", "string",
```

```
"description", "地区筛选: 华东、华南、华北、西南、西北, 不填则查询全国",
"enum", List.of("华东", "华南", "华北", "西南", "西北")
));
properties.put("period", Map.of(
    "type", "string",
    "description", "时间段: 本月、上月、本季度、上季度、本年, 默认本月",
    "default", "本月"
));
properties.put("queryType", Map.of(
    "type", "string",
    "description", "查询类型: summary(汇总)、ranking(排名)、detail(明细)、trend(趋势)",
    "default", "summary"
));
// ...

return Tool.builder()
    .name("sales_query")
    .description("查询软件销售数据, 支持按地区、时间、产品、销售人员等维度筛选")
    .inputSchema(new JsonSchema("object", properties, List.of(), null, null, null))
    .build();
}
```

工具有 6 个参数，但用户只说了一句“华东区本月的销售额是多少”。参数提取器需要从这句话里抽出 region、period、queryType，其余参数填默认值。这个提取过程是怎么做的？下一篇讲。

核心类速查表

类名	职责	关键方法
IntentKind	意图类型枚举 (KB / SYSTEM / MCP)	fromCode()
IntentNode	意图节点, MCP 节点携带 mcpToolId	isMCP(), getMcpToolId()
NodeScoreFilters	按类型过滤意图	mcp(), kb()
McpToolRegistry	工具注册表接口	register(), getExecutor()
DefaultMcpToolRegistry	注册表默认实现, Map 存储	init() 自动注册
McpClientAutoConfiguration	启动时连接 MCP Server 并注册远端工具	registerRemoteTools()
McpClientProperties	MCP Server 连接配置	getServers()
McpToolExecutor	工具执行器接口	execute(), getToolDefinition()
McpClientToolExecutor	远程调用执行器实现	execute() 封装异常处理
McpParameterExtractor	参数提取器接口 (下一篇展开)	extractParameters()
RetrievalEngine	检索引擎, 协调 KB 和 MCP	executeSingleMcpTool(), executeMcpTools()
ContextFormatter	上下文格式化器	formatMcpContext()
RetrievalContext	KB + MCP 结果的统一载体	hasMcp(), hasKb()

小结与下一篇预告

本篇完整走了一遍 MCP 工具调用在问答流水线中的链路，核心要点：

- MCP 嵌在阶段 6 retrieve 内部，和 KB 检索并行执行，不是独立流水线。分流发生在 buildSubQuestionContext() 的前两行，NodeScoreFilters.mcp() 把 kind=MCP 且 mcpToolId 非空的意图节点过滤出来
- McpToolRegistry 是工具注册表，内部是一个 Map<String, McpToolExecutor>。启动时由 McpClientAutoConfiguration 连接配置的 MCP Server，通过 listTools() 发现远端工具并注册。运行时通过 toolId 一查就能找到执行器

3. `McpClientToolExecutor` 通过 MCP 官方 SDK 的 `McpSyncClient.callTool()` 发起远程调用。异常统一包装成 `isError=true` 的 `CallToolResult`，不上抛
4. 异常处理有两层保护——`McpClientToolExecutor` 内部 `catch` + `executeMcpTools()` 外层 `catch`。单个工具失败不影响其他工具，不拖垮整个问答流程
5. 多个 MCP 意图通过 `mcpBatchExecutor` 线程池并行执行，结果按 `toolId` 分组
6. `ContextFormatter.formatMcpContext()` 将工具结果格式化为带 `<data>` 标签的文本，区分成功和失败结果。意图节点上的 `promptSnippet` 会作为 `<rules>` 一起嵌入上下文
7. 最终通过 `RetrievalContext` 的 `mcpContext` 字段和 KB 的 `kbContext` 一起传给 Prompt 组装。`hasMcp()` 会影响 LLM 的 `temperature` (0.3 vs 0)

整个链路里有一个环节被有意跳过了——参数提取。`executeSingleMcpTool()` 里调了

`mcpParameterExtractor.extractParameters(question, tool, customParamPrompt)`，传入的是用户的自然语言，返回的是工具需要的结构化参数 Map。就拿 `sales_query` 来说，工具定义了 `region`、`period`、`queryType` 等 6 个参数，用户可只说了一句“华东区本月销售额多少”。这中间的转换怎么做的？下一篇来拆参数提取器。

